Book Metadata API Report

Submitted Materials:

GitHub repository: https://github.com/IvenMilesMaac/book-metadata-api

API Documentation: https://github.com/IvenMilesMaac/book-metadata-api/blob/main/api-technical-report.pdf

Presentation Slides: https://docs.google.com/presentation/d/1lRt8NgHjRO4XJUoCm1vvyUxjCCBRByebVfc2KF1Omdk/edit?usp=sharing

Introduction

The book metadata domain was selected because the dataset provides multiple quantitative fields per record. Fields like *average_rating* and *ratings_count* naturally support the implementation of analytical endpoints beyond basic CRUD, such as filtering by top-rated or most popular books. This allowed for a more meaningful API that demonstrates data-driven querying than simple record retrieval.

The dataset was sourced from Kaggle (Soumik, 2019) and contains approximately 11,123 book records. The fields used were title, average_rating, ratings_count, and authors. Additional fields present in the dataset such as ISBN, language code, number of pages, and number of text reviews were omitted as they were outside the scope of this project.

Technology Stack

FastAPI was chosen as the web framework for this project due to its automatic generation of interactive Swagger UI documentation at */docs*, which significantly streamlined endpoint testing during development. Its built-in integration with Pydantic also provides automatic request validation and meaningful error responses without requiring additional code. Compared to other options like Django, FastAPI requires considerably less setup and configuration overhead, making it more suitable for a lightweight API-focused application.

SQLite was chosen as the database due to its simplicity and zero-configuration setup, as it requires no separate database server and is built into Python's standard library. While a production-scale API would likely benefit from PostgreSQL for improved

concurrency and scalability, SQLite is appropriate here given the read-heavy nature of the API and the scale of the dataset. **SQLAlchemy** was used as the ORM, allowing database interactions to be written entirely in Python rather than raw SQL. This keeps the codebase consistent and reduces the risk of SQL injection vulnerabilities.

For deployment, I chose **Render** (Render, 2026) due to its free tier availability and its integration with GitHub, which automatically redeploys the application whenever changes are pushed to the repository. This made the development and deployment workflow seamless, as no manual server configuration or separate deployment steps were required.

Architecture & Design

Project Structure

```
app/
|----- main.py
|----- database.py
|----- models.py
|----- schemas.py
|----- crud.py
|----- routes/
         |-------- books.py
         |-------- authors.py
         |-------- categories.py
```

The project follows a layered architecture, separating concerns across distinct modules. The application entry point is **main.py**, which initialises the FastAPI application and registers the route files. Database connection and session management are handled in **database.py**. Database table definitions are contained in **models.py**, whilst **schemas.py** defines the Pydantic models that control data validation and serialisation for API inputs and outputs. All database operations are centralised in **crud.py**, and the API endpoints are defined across separate route files grouped by entity. This separation ensures the database logic, validation logic, and routing logic remain independent, making the codebase easier to maintain and debug.

Database Design

The database consists of 3 entities: Book, Author, and Category. The Book entity stores the core metadata fields used by the API, including *title, average_rating*, *and ratings_count*, the latter two of which enable the analytical endpoints. Books have

many-to-many relationships with both Authors and Categories, as a singel book can have multiple authors and belong to multiple categories. These relationships are handled through association tables (*book_author* and *book_category*) rather than foreign keys directly on the Book table. An ER diagram illustrating these relationships is shown in Figure 1.

Figure 1. Entity Relationship Diagram for the Book Metadata API Database

A key design decision was the use of separate Pydantic schemas for different operations rather than a single shared model. *BookCreate* defines the fields accepted when creating a book, *BookUpdate* makes all fields optional to support partial updates, and *BookRead* defines the response structure returned to the client. This separation ensures that internal database details are not unintentionally exposed in API responses, and that input validation rules are enforced consistently. For example, *BookCreate* accepts *author_ids* and *category_ids* as lists of integers, whilst *BookRead* returns fully nested *author_objects*, giving the client more useful information without requiring additional requests.

Testing & Error Handling

Endpoints were tested throughout development using the automatically generated Swagger UI interface at */docs*, which allowed individual endpoints to be executed, and their responses inspected interactively without requiring any additional tooling. This was particularly useful during implementation for verifying request validation and checking that relationships between entities were being resolved correctly. Following implementation, automated testing was carried out using a dedicated test.py script which programmatically tested each endpoint, verifying correct status codes and response structures. This two-stage approach ensured that both individual endpoint behaviour and overall API functionality were validated.

The test suite, implemented using FastAPI's *TestClient*, runs tests directly against the application without requiring a live server. Tests cover both successful operations and expected failure cases – for example, verifying that requests to non-existent resources return a 404 response, and that requests to protected endpoints without a valid API key return a 401 response. More complex relational operations were also tested, such as creating an author and a book in sequence and verifying that filtering books by that

author returns the correct results. A limitation of the test suite is that category-related endpoints are not covered, reflecting the absence of category data in the database.

Challenges & Lessons Learned

As this project involved several tools and frameworks that had not been used previously, many of the challenges encountered stemmed from inexperience rather than technical complexity. FastAPI and SQLAlchemy were both used for the first time, requiring an understanding of how routing, schema validation, and ORM-based database interactions work in practice. Through working with these tools, a clearer understanding was developed of how Pydantic schemas control data flow between the client and the database, and how SQLAlchemy's relationship model handles complex associations such as many-to-many relationships.

Deploying the API to a live server via Render was also a new experience, requiring familiarity with environment variable configuration and cloud build processes. Writing automated tests using FastAPI's *TestClient* was similarly unfamiliar but developed an appreciation for the importance of testing both successful operations and expected failure cases rather than only verifying the happy truth.

Limitations & Future Development

The primary limitation of the API is that the Goodreads dataset used did not include category data, meaning the category entity and its related endpoints are implemented and documented but cannot be fully utilised. This also meant that category-related endpoints could not be included in the automated test suite. Additionally, whilst *skip* and *limit* parameters are supported on list endpoints, no pagination metadata is returned to the client, such as total record count or page number, which limits the usefulness of pagination for client applications. A further limitation is that SQLite, whilst appropriate for a project of this scale, is not suitable for production use due to its handling of concurrent write requests. Finally, there are no uniqueness constraints preventing duplicate records, meaning books with identical titles or authors with identical names can be created without restriction.

Several areas could be explored in future development. The most immediate improvement would be sourcing a dataset that includes category data, which would allow the category entity to be fully populated, and its endpoints properly tested. Migrating the database from SQLite to PostgreSQL would improve scalability and make the API more suitable for production deployment. Pagination could be improved by returning metadata such as total record count alongside results. The analytical

capabilities of the API could also be extended, like genre tend analysis. Uniqueness constraints on book title and author names would also improve data integrity.

GenAI Declaration & Analysis

Instance	**Purpose**
Initial project structure	Generated initial source files (*models.py, schemas.py, crud.py, routes/*) as a starting scaffold.
Dataset import script	Generated a Python script to parse the Goodreads CSV and import records into the SQLite database
min_ratings parameter	Discussed the insight that sorting by *average_rating* alone is misleading without a minimum *ratings_count* threshold and implemented this as a query parameter on the */book/top-rated/* endpoint.
API documentation	Generated the full API documentation iteratively by providing actual source files (crud.py, schemas.py, models.py) to ensure accuracy, correcting outputs against real API behaviour.
Debugging	Diagnosed a 500 Internal Server Error by providing server logs and source code, identifying typos in *get_category* and a Pydantic v2 serialisation error
Technical report	Used to structure and draft sections of the technical report, with content verified and personalised throughout.

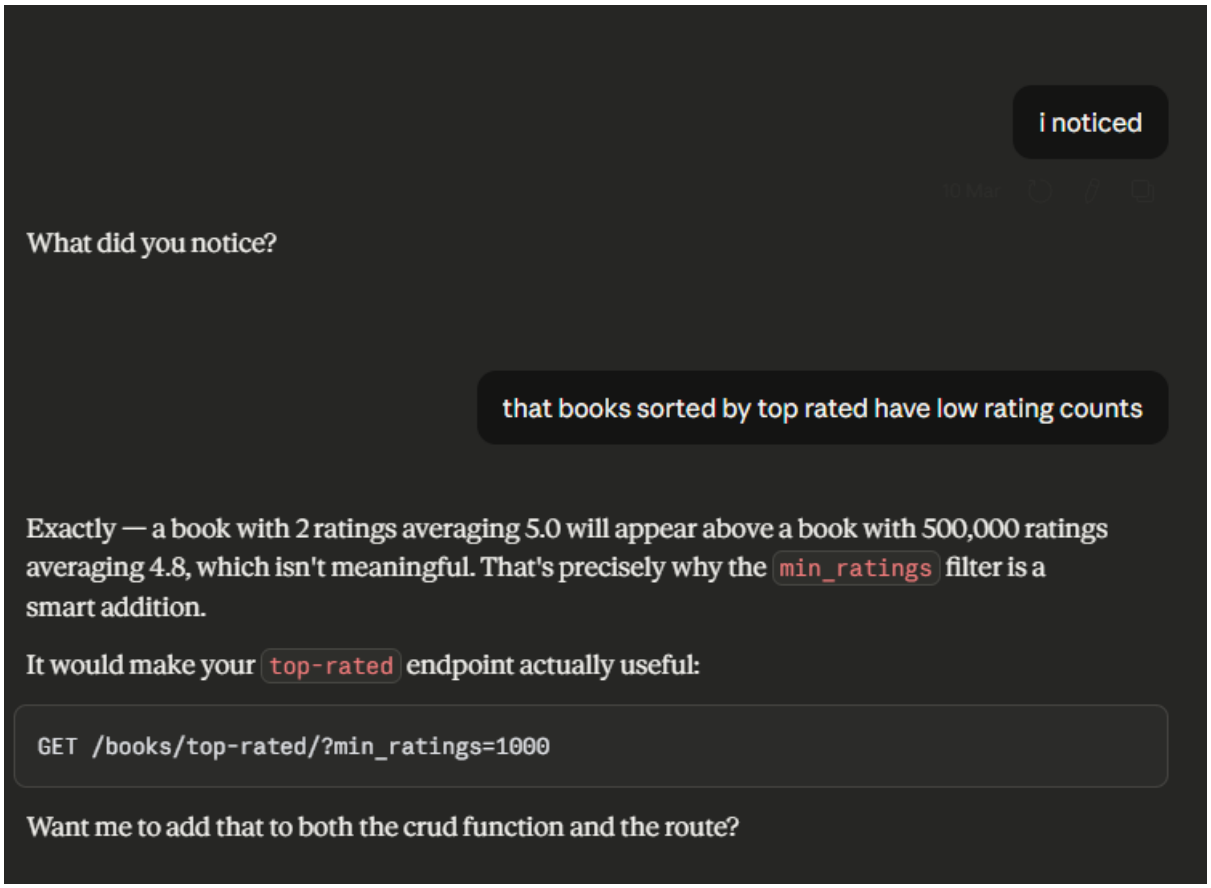
Throughout this project, Generative AI was used as an iterative development and documentation partner rather than a tool for blind code generation. Where possible, actual source files and server logs were provided to ensure responses were accurate and context-specific rather than generic. The most significant example of this approach was the API documentation, which was refined across multiple exchanges by providing real schema and CRUD files, correcting field names, data types, and response structures against actual API behaviour. This methodology ensured that AI outputs were critically evaluated and verified rather than accepted uncritically, reflecting a deliberate and methodologically sound approach to AI-assisted development (Anthropic, 2026).

References

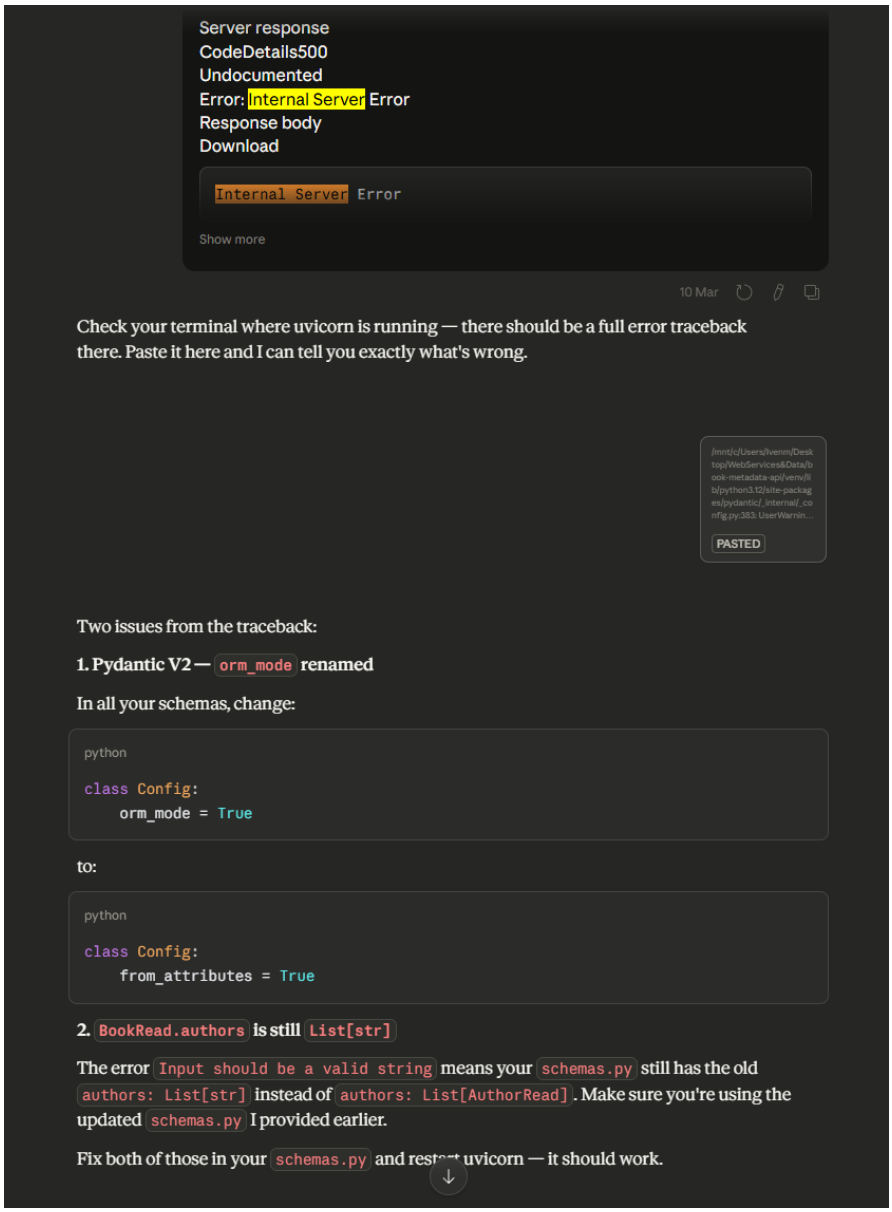
- Anthropic. 2026. Claude (claude-sonnet-4-6) [Large language model]. [Online]. [Accessed: 11 March 2026]. Available from: https://claude.ai
- Render. 2026. Render – Cloud Application Hosting. [Online]. [Accessed: 11 March 2026]. Available from: https://render.com
- Soumik. 2019. Goodreads Books [Dataset]. Kaggle. [Online]. [Accessed: 11 March 2026]. Available from: https://www.kaggle.com/datasets/jealousleopard/goodreadsbooks/data

Appendix

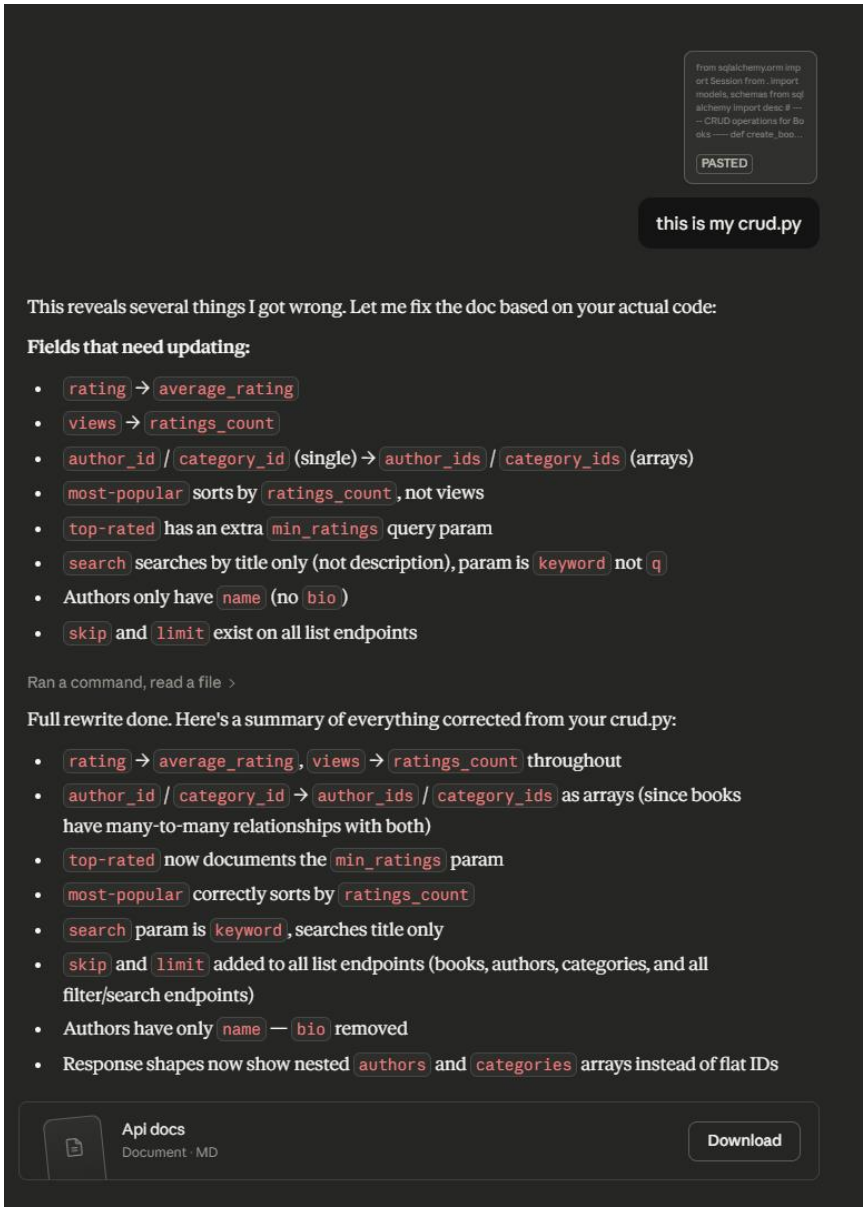
Appendix A: AI Conversation Log – min_ratings Parameter Discussion



Appendix B: AI Conversation Log – Debugging 500 Internal Server Error and Pydantic Serialisation Error



Appendix C: AI Conversation Log – Documentation Correction from crud.py



Appendix D: AI Conversation Log – Categories Endpoint Judgement Call

what would be suitable for my coursework brief

I'd need to see the brief to give you a proper answer — can you paste it here?

Based on the brief, **keep the endpoint but add a note** explaining it returns an empty list when no categories exist. Here's why:

- The brief explicitly requires documentation to cover **all available endpoints** — removing it would be a gap
- The marking criteria rewards **effective error handling** and **comprehensive documentation** — noting the empty list behaviour is exactly that
- It also shows honest, accurate documentation which examiners will appreciate during Q&A

Ran a command, read a file >

Done. One wider observation from reading the brief — your documentation needs to be submitted as a **PDF** referenced in your README. Once you're happy with the `.md` file in VS Code, you'll need to export it to PDF. The easiest way is the **Markdown PDF** VS Code extension — it converts in one click. Want me to help with any of the other deliverables too (README, technical report, slides)?

Api docs
Document · MD
Download